

C++ & Qt AI 사용 보고서(Day3)

학번: 2026406076

이름: 정창규

제출일: 2026-09-07

1. AI 사용 목적

이번 Day3 과제를 진행하면서 수업 자료를 먼저 확인한 뒤 이해가 잘 되지 않는 개념이나 구현 과정에서 막히는 부분을 AI에 질문하였다. 과제의 답을 바로 얻기보다는 Dijkstra와 A* 알고리즘의 차이, Qt에서 UI를 구성하는 방법, 코드에서 오류가 발생한 이유 등을 확인하는 용도로 주로 사용하였다.

특히 알고리즘을 공부할 때에는 단순히 코드만 확인하기보다 각 값이 어떤 의미를 가지는지 질문하였다. 구현 과정에서도 한 번에 전체 프로그램을 완성하기보다는 프로젝트 구조 설정, UI 구성, 기능 추가, 오류 수정 순서로 나누어 진행하면서 필요한 부분을 질문하였다.

2. 알고리즘 개념 학습에서 AI 활용

Dijkstra 알고리즘을 공부하면서 처음에는 각 간선의 비용과 시작점에서 각 노드까지의 누적 비용을 구분하는 부분이 헷갈렸다. 그래서 AI에 예시 그래프를 기준으로 각 값이 어떤 의미인지 질문하였다.

AI의 설명을 통해 Dijkstra 알고리즘에서는 단순히 현재 노드와 다음 노드 사이의 간선 비용만 비교하는 것이 아니라, 시작점에서 현재 위치까지 이동한 누적 비용을 기준으로 더 짧은 경로가 있는지 확인한다는 점을 이해할 수 있었다.

예를 들어 어떤 노드까지 직접 이동하는 비용과 다른 노드를 거쳐 이동하는 비용을 비교하여 더 작은 값이 있으면 해당 값으로 갱신하는 과정을 확인하였다. 이를 통해 Dijkstra 알고리즘이 시작점에서 각 노드까지의 최소 비용을 계속 갱신하면서 진행된다는 것을 이해하였다.

A* 알고리즘에서는 $g(n)$, $h(n)$, $f(n)$ 의 역할을 구분하는 데 AI를 활용하였다. 처음에는 Dijkstra와 A*가 비슷하게 느껴졌지만 질문을 통해 두 알고리즘이 다음과 같은 기준으로 탐색한다는 것을 정리할 수 있었다.

- $g(n)$: 시작점에서 현재 노드까지 이동한 실제 비용
- $h(n)$: 현재 노드에서 도착점까지 남은 비용을 예상한 값
- $f(n)$: $g(n) + h(n)$ 으로 계산한 값

특히 $h(n)$ 이 휴리스틱 함수라는 점과 A*가 이 값을 이용하여 도착점 방향을 고려하면서 탐색한다는 내용을 이해하였다.

이후 Task2 프로그램을 실행하면서 같은 맵에서 Dijkstra와 A*의 결과를 비교하였다. 두 알고리즘의 최종 경로 비용은 같았지만 A*의 탐색 노드 수가 더 적게 나타나는 경우를 확인하였다. 이전에 공부한 $h(n)$ 의 역할과 실제 실행 결과를 연결해서 확인하면서 두 알고리즘의 차이를 더 쉽게 이해할 수 있었다.

A*에서 $h(n)$ 이 휴리스틱 함수야? $g(n)$ 이랑 $h(n)$, $f(n)$ 이 각각 뭐고 어떻게 다른지도 설명해줘.

응. A*에서 $h(n)$ 이 휴리스틱 함수가 맞아.

쉽게 정리하면:

- $g(n)$: 시작점에서 현재 노드 n 까지 실제로 이동한 비용
- $h(n)$: 현재 노드 n 에서 도착점까지 남은 비용을 예상한 값
- $f(n)$: A*가 어떤 노드를 먼저 탐색할지 판단할 때 사용하는 값

A*에서는

```
</> 일반 텍스트
f(n) = g(n) + h(n)
```

으로 계산해.

예를 들어 시작점에서 현재 위치까지 실제 비용이 5이고, 현재 위치에서 도착점까지 예상 비용이 3이면

```
</> 일반 텍스트
g(n) = 5
h(n) = 3
f(n) = 8
```

이 되는 거야.

Dijkstra와 비교하면 더 쉽게 볼 수 있어.

```
</> 일반 텍스트
Dijkstra → g(n)을 기준으로 탐색
A*       → g(n) + h(n)을 기준으로 탐색
```

그래서 A*는 $h(n)$ 을 이용해서 도착점 방향을 어느 정도 고려해서 탐색할 수 있어. 적절한 휴리스틱 함수를 사용하면 Dijkstra보다 탐색해야 하는 노드 수를 줄일 수 있어

ChatGPT에 메시지 보내기

Dijkstra에서 다음 노드를 고를 때 간선 하나의 비용을 보는 게 아니라 시작점부터 지금까지의 누적 비용을 보는 거야?

응. Dijkstra에서는 간선 하나의 비용만 보는 게 아니라 시작점에서 현재 노드까지의 누적 비용을 기준으로 다음 노드를 선택해.

예를 들어 시작점이 1번이고,

```
</> 일반 텍스트
1 → 4 비용 = 1
4 → 5 비용 = 1
```

이면 5번 노드까지의 비용은 간선 하나의 값인 1이 아니라

```
</> 일반 텍스트
1 + 1 = 2
```

가 돼.

그리고 만약 1번에서 5번으로 바로 가는 경로의 비용이 5라면,

```
</> 일반 텍스트
직접 이동 : 5
1 → 4 → 5 : 2
```

이므로 더 작은 값인 2를 사용해.

즉 Dijkstra는 각 노드에 대해

```
</> 일반 텍스트
시작점 → 현재 노드까지의 최소 누적 비용
```

을 계속 비교하고 갱신하면서 탐색하는 알고리즘이라고 보면 돼

📄 🔄 📌 ...

3. Task2 구현 과정에서 AI 활용

Task2에서는 Qt를 이용해 Dijkstra와 A*를 비교하는 프로그램을 구현하면서 AI를 활용하였다. 프로젝트 파일을 include, src, ui 폴더로 나눈 뒤 ui_mainwindow.h file not found 오류가 발생했는데, AI에 원인을 질문하여 CMake에서 ui 폴더의 경로를 추가해야 한다는 점을 확인하고 오류를 해결하였다.

또한 QGridLayout과 QPushButton을 이용해 행과 열 크기에 맞는 격자 맵을 만드는 방법을 질문하였다. 이후 장애물, 시작점, 도착점을 직접 설정하고 Dijkstra와 A*를 실행할 수 있도록 기능을 하나씩 추가하였다.

두 알고리즘을 같은 맵에서 실행했을 때 A*의 탐색 노드 수가 더 적게 나타나 정상적인 결과인지 AI에 다시 질문하였다. 이를 통해 A*가 휴리스틱 값을 이용해 도착점 방향을 고려하기 때문에 탐색 범위를 줄일 수 있다는 점을 실제 결과와 연결해서 이해할 수 있었다.

Qt에서 행과 열 값을 받아서 QPushButton으로 격자 맵을 동적으로 만들려면 어떻게 해야 돼?

Qt에서는 행과 열 값을 SpinBox에서 받아서 그 크기만큼 QPushButton을 반복해서 만들고, QGridLayout에 넣으면 돼.

예를 들어 `rowSpinBox`, `colSpinBox` 에서 값을 받아서 `gridWidget` 안에 격자를 만든다고 하면 이런 식이야.

</> C++

```
void MainWindow::createMap()
{
    int rows = ui->rowSpinBox->value();
    int cols = ui->colSpinBox->value();

    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {
            QPushButton *button = new QPushButton(ui->gridWidget);

            button->setText("");
            button->setMinimumSize(20, 20);

            gridLayout->addWidget(button, row, col);
        }
    }
}
```

여기서

</> C++

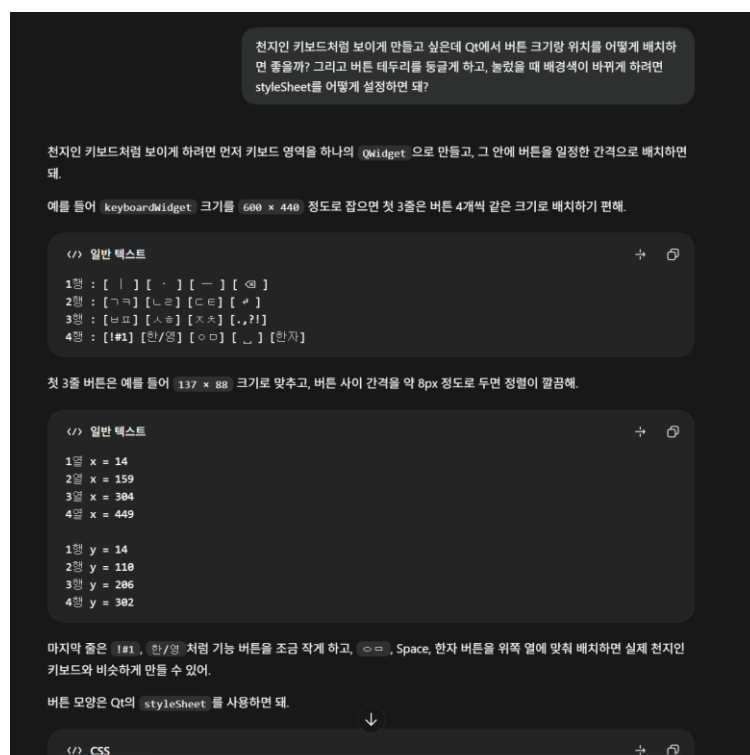
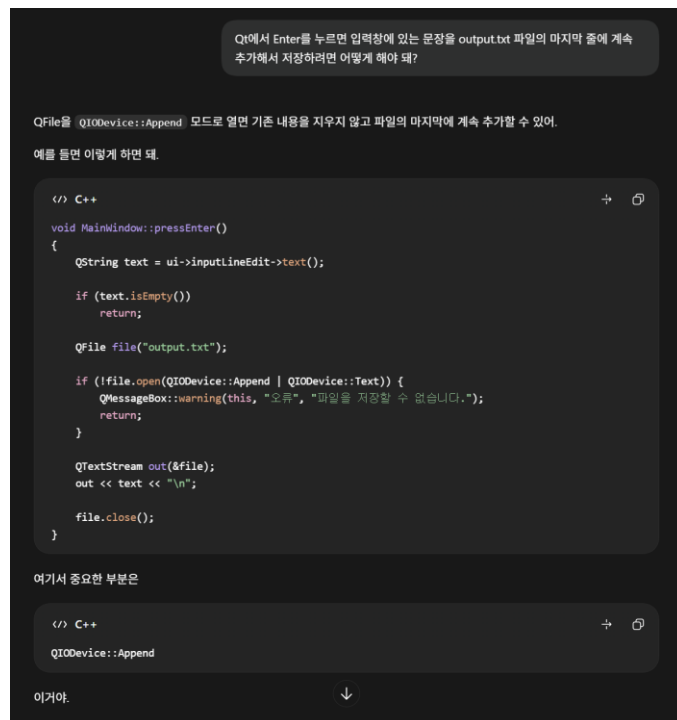
```
int rows = ui->rowSpinBox->value();
int cols = ui->colSpinBox->value();
```

이 부분에서 사용자가 입력한 행과 열 값을 가져오고,

</> C++

4. Task3 구현 과정에서 AI 활용

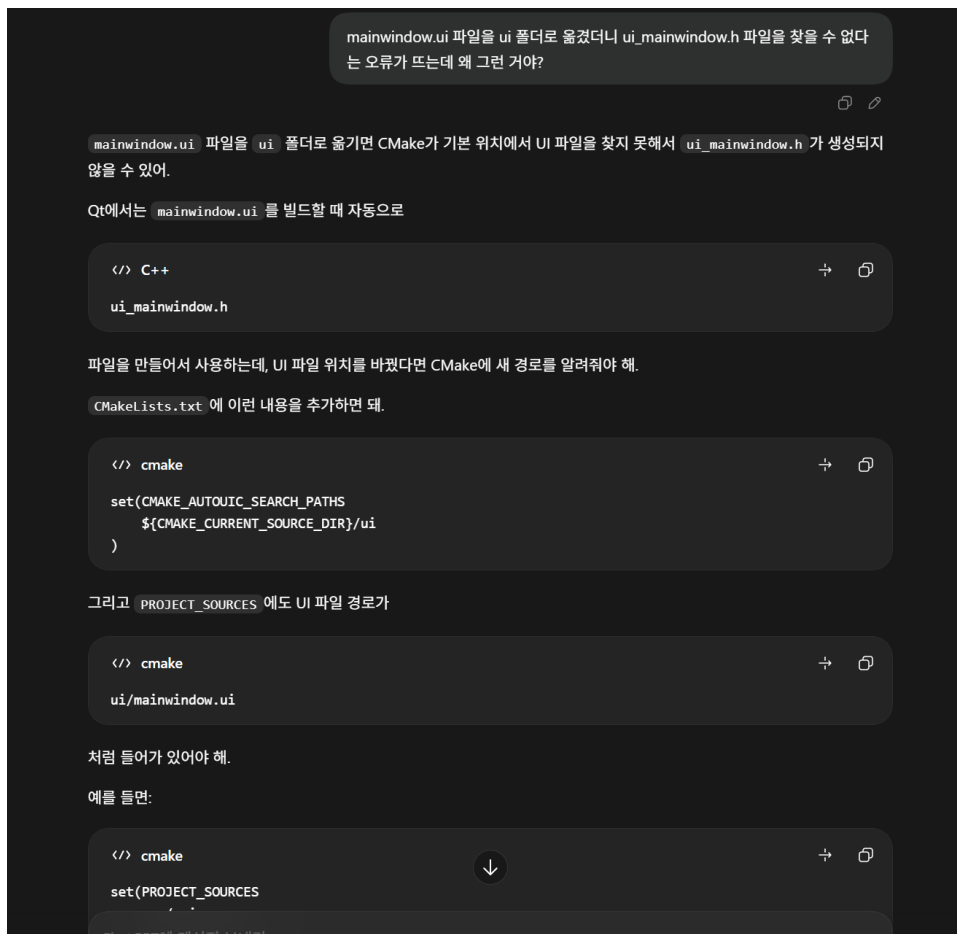
Task3에서는 천지인 키보드 형태의 UI를 만들고, 한/영 전환과 대소문자, 숫자 및 특수문자 입력 기능을 구현하는 과정에서 AI를 활용하였다. 버튼의 위치와 크기를 정하거나 실제 천지인 키보드와 비슷하게 보이도록 배치하는 과정에서도 참고하였다. 또한 Enter를 눌렀을 때 입력한 문장이 output.txt에 추가되도록 파일 입출력 기능을 구현하고, 저장된 파일의 위치를 확인하는 방법을 질문하였다.



5. 오류 해결 과정에서 AI 활용

과제를 진행하면서 오류가 발생했을 때 해결 방법만 확인하기보다 왜 오류가 생겼는지를 같이 질문하였다. 가장 기억에 남는 것은 Task2에서 파일 구조를 변경한 뒤 발생한 `ui_mainwindow.h` file not found 오류였다. UI 파일을 `ui` 폴더로 옮겼지만 CMake에서 해당 위치를 찾지 못한 것이 원인이었고, 경로를 다시 설정한 뒤 CMake를 재실행하여 해결하였다.

Task3에서는 Enter를 눌렀을 때 문장이 사라져 처음에는 저장되지 않은 것으로 생각했지만, 실제로는 `output.txt`에 저장된 뒤 입력창이 초기화되는 동작이었다. AI에 파일 위치를 확인하는 방법을 질문하여 터미널에서 저장된 내용을 직접 확인하였다.



6. AI를 사용하며 느낀 점

이번 과제를 진행하면서 AI는 코드를 바로 받는 것보다 이해되지 않는 개념이나 오류가 발생한 이유를 확인할 때 더 도움이 된다고 느꼈다. 특히 Dijkstra와 A*의 차이처럼 처음에는 비슷하게 느껴졌던 내용을 여러 번 나누어 질문하면서 각 알고리즘이 어떤 기준으로 탐색하는지 이해할 수 있었다.

Qt 구현에서도 한 번에 전체 코드를 완성하기보다 UI 구성, 기능 연결, 오류 수정 순서로 나누어 질문하는 방식이 더 이해하기 쉬웠다. 코드를 그대로 사용하는 것보다 각 기능이 어떤 역할을 하는지 확인하면서 진행해야 이후에 수정할 부분이 생겨도 대응하기 쉽다는 점도 알게 되었다.